# 风险变量查询服务（fxbl）· API 接口文档与使用手册

> 面向接入方（客户）技术与运维人员的对外接口说明。
> 版本：fxbl（风险变量）｜通信：HTTP + JSON｜编码：UTF-8
> 说明：本服务采用统一的请求信封与 MD5 加签方式，响应分为 `head`（网关头部）与 `body`（业务结果）两部分。其余内部处理细节调用方无需关心。
> 关键特性：风险变量的查询结果是一个结构化对象（规则决策结果 + 多维度申请行为变量），本服务将其原样序列化为 JSON 字符串置于 `body.result.range`，由调用方自行解析（见 3.1.5 result.range 结构说明）。
> 计费提示：本服务按有效查询次数计费——查得数据（ `001` ）与查无结果（ `999` ）**均计费**，详见「五、计费说明」。

## 一、接入必读

### 1.1 适用范围

本文档适用于接入本平台「风险变量查询服务（fxbl）」的第三方产品技术开发人员与日常运维人员。

### 1.2 接入须知

1. 正式服务地址（任选其一，两域名等价）： `http://aiszcloud.cn:8080` 、 `http://aiszcloud.com.cn:8080` 。
2. 接入前需先申请开通账户，由我方分配 `appKey` 与 `appSecret` （加签密钥）。
3. 接入时需向我方提供贵司的外网出口 IP，以便加入访问白名单。

### 1.3 接口说明

| 项目 | 说明 |
| --- | --- |
| 请求方式 | POST（计费查询数查询为 GET） |
| 通信协议 | HTTP |
| 数据格式 | 请求体与响应体均为 JSON |
| 字符编码 | UTF-8 |
| 超时时间 | 4 秒（建议客户端读超时 ⩾ 5 秒） |
| HTTP 状态码 | 恒为 200；业务结果与错误均通过响应体的 `head.errorCode` / `body.code` 表达 |
| 签名 | 调用时需对 `body` 中的业务参数 + 我方分配的 `appSecret` 进行 MD5 加签，详见「二、鉴权与加签」 |

### 1.4 环境说明

- 正式环境：使用正式账户，调用已开通接口，按有效查询次数计费（见「五、计费说明」）。
- 测试环境：使用测试账户，返回挡板/联调数据。
- 正式账户仅适用于正式环境，测试账户仅适用于测试环境。

## 二、鉴权与加签

所有业务接口共用同一套请求信封与鉴权方式。

### 2.1 请求信封（顶层参数）

| 参数 | 示例 | 类型 | 必填 | 说明 |
| --- | --- | --- | --- | --- |
| appKey | y890fxbl | String | 是 | 我方分配给客户的公开标识 |
| sign | 0528999dd55c025b8f36fc72dceb1f63 | String | 是 | 对 body 业务参数的 MD5 签名（见 2.3） |
| encryptionType | 1 | int | 否 | 参数加密类型，1 = 明文（默认） |
| body | {...} | Object | 是 | 业务请求体，见各接口定义 |

注意：本服务不需要 `apiKey` 参数（产品由请求路径区分）。`appKey` / `sign` / `encryptionType` 不参与签名计算。调用方传入的 `body` 字段一律为明文。

### 2.2 鉴权校验顺序

网关按以下顺序校验，任一失败立即返回对应 `head.errorCode`（直接拒绝，不计费）：

1. `appKey` 是否存在 → 否则 `505001`
2. `appKey` 是否匹配到账户 → 否则 `505004`
3. 账户是否有效（启用且在有效期内）→ 否则 `505007`
4. 签名是否正确 → 否则 `505002`

### 2.3 加签方式

1. 取出 `body` 中所有非空的业务参数（不含文件/字节流类型，不含值为空的参数）。
2. 按参数名（key）的 ASCII 升序排序；首字符相同则依次比较后续字符。
3. 将排序后的参数按 `参数名参数值` 直接拼接，最后追加 `appSecret`，得到待签名串。
4. 对待签名串做 MD5，取 32 位小写十六进制字符串，赋值给 `sign`。

示例：`body = { "name": "张三", "idCard": "360429199806073112", "mobile": "17325421010" }`，`appSecret = "<你的密钥>"`。

按 ASCII 升序排序后 key 顺序为 `idCard → mobile → name`，待签名串为：

```
idCard360429199806073112mobile17325421010name张三<你的密钥>
```

`sign = MD5(待签名串)` 的小写十六进制值。

提示：拼接顺序由 key 的 ASCII 决定，请勿写死字段顺序；新增字段时排序会自动变化。

## 2.4 加签代码示例

Java

```
public static String sign(Map<String, String> bodyParams, String appSecret) throws
    StringBuilder sb = new StringBuilder();
    List<String> keys = new ArrayList<>(bodyParams.keySet());
    Collections.sort(keys); // ASCII 升序
    for (String k : keys) {
        String v = bodyParams.get(k);
        if (v == null || v.isEmpty()) continue; // 剔除空值
        sb.append(k).append(v);
    }
    sb.append(appSecret);
    MessageDigest md = MessageDigest.getInstance("MD5");
    byte[] digest = md.digest(sb.toString().getBytes(StandardCharsets.UTF_8));
    StringBuilder hex = new StringBuilder();
    for (byte b : digest) hex.append(String.format("%02x", b));
    return hex.toString();
}
```

Python

```
import hashlib

def sign(body_params: dict, app_secret: str) -> str:
    parts = []
    for k in sorted(body_params.keys()):   # ASCII 升序
        v = body_params[k]
        if v is None or v == "":
            continue                        # 剔除空值
        parts.append(f"{k}{v}")
    raw = "".join(parts) + app_secret
    return hashlib.md5(raw.encode("utf-8")).hexdigest()  # 小写 hex
```

Go

```
func sign(body map[string]string, appSecret string) string {
    keys := make([]string, 0, len(body))
    for k, v := range body {
        if v != "" { // 剔除空值
            keys = append(keys, k)
        }
    }
    sort.Strings(keys) // ASCII 升序
    var sb strings.Builder
    for _, k := range keys {
        sb.WriteString(k)
        sb.WriteString(body[k])
    }
    sb.WriteString(appSecret)
    sum := md5.Sum([]byte(sb.String()))
    return hex.EncodeToString(sum[:]) // 小写 hex
}
```

## 三、接口列表

### 3.1 风险变量查询（fxbl）

| 项目 | 内容 |
| --- | --- |
| 路径 | `POST /v1/openapi/zlx/querySrmxFXBL` |
| 完整地址 | `http://aiszcloud.cn:8080/v1/openapi/zlx/querySrmxFXBL`（或 `http://aiszcloud.com.cn:8080/v1/openapi/zlx/querySrmxFXBL`） |
| 鉴权 | appKey + MD5 签名（见第二章） |

#### 3.1.1 请求 body 参数

| 参数 | 示例 | 类型 | 必填 | 说明 |
| --- | --- | --- | --- | --- |
| name | 张三 | String | 是 | 姓名，明文传入，不能为空 |
| idCard | 360429199806073112 | String | 是 | 身份证号（18 位，末位可为 `X`），明文传入 |
| mobile | 17325421010 | String | 是 | 手机号（11 位，以 1 开头），明文传入 |

本接口 `name`、`idCard`、`mobile` 三项**均必填**；额外传入的字段不参与校验、亦不影响结果。参数缺失或格式非法（姓名为空、身份证号或手机号不符）将返回 `head.errorCode = 505062`，直接拒绝，不计费。

### 3.1.2 请求示例

```
{
  "encryptionType": 1,
  "appKey": "y890fxbl",
  "sign": "0528999dd55c025b8f36fc72dceb1f63",
  "body": {
    "name": "张三",
    "idCard": "360429199806073112",
    "mobile": "17325421010"
  }
}
```

### 3.1.3 响应结构

响应分为 `head`（网关头部）与 `body`（业务结果）两部分：

head 字段：

| 参数 | 示例 | 类型 | 说明 |
| --- | --- | --- | --- |
| errorCode | 0 | String | 网关返回码。`0` = 成功（含查得/查无）；非 0 = 网关级错误，此时无 body |
| errorMsg | success | String | 返回描述 |
| logId | a1b2c3... | String | 全链路追踪 ID，排障/对账时请提供 |
| time | 128 | Number | 服务处理耗时（毫秒） |
| timestamp | 1718456789012 | Number | 响应时间戳（毫秒） |

body 字段（仅 `head.errorCode = 0` 时返回）：

| 参数 | 示例 | 类型 | 说明 |
| --- | --- | --- | --- |
| code | 001 | String | 业务结果码。`001` = 查得数据【计费】；`999` = 查无结果【计费】 |
| msg | 成功 | String | 业务描述 |
| reqid | lkf9x2... | String | 本次请求流水号 |
| uid | lkf9x2... | String | 交易流水号（对账用） |
| result | {...} | Object | 业务内容，仅 `code = 001` 时存在 |
| result.range | "{...}" | String | 风险变量结果对象的 JSON 字符串，调用方需 `JSON.parse` 后使用，结构见 3.1.5 |

### 3.1.4 响应示例

① 查得数据（计费）


```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d4", "time": 1
  "body": {
    "code": "001",
    "msg": "成功",
    "reqid": "lkf9x2ab",
    "uid": "lkf9x2ab",
    "result": {
      "range": "{\"Rule_final_decision\":\"Accept\",\"Rule_final_weight\":\"60\",\"
    }
  }
}
```


② 查无结果（计费）


```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d6", "time": 9
  "body": {
    "code": "999",
    "msg": "查无结果",
    "reqid": "lkf9x2ad",
    "uid": "lkf9x2ad"
  }
}
```

`999` 表示本次查询已完成、库中确无该主体的可用记录——这是一个有效的查询结论，与 `001` 同样计费（见「五、计费说明」）。

③ 网关级错误（无 body）

```
{
  "head": { "errorCode": "505002", "errorMsg": "账号信息异常", "logId": "a1b2c3d7", "
}
```

### 3.1.5 result.range 结构说明

`result.range` 是风险变量结果对象序列化后的 JSON 字符串（完整结果原样返回，未做裁剪）。调用方应先对该字符串做一次 `JSON.parse` / 反序列化，再按字段名读取。**所有字段值均为字符串**（数值以字符串形式给出，如 `"2.75"`），仅返回有统计值的字段。字段分为「规则决策字段」与「申请行为变量」两类。

规则决策字段：

| 字段 | 含义 |
| --- | --- |
| `Rule_final_decision` | 最终决策结果：`Accept` 通过 / `Reject` 拒绝 / `Review` 复议 |
| `Rule_final_weight` | 最终规则评分 |
| `Rule_name_QJF076` / `Rule_weight_QJF076` | 命中规则「在非银机构申请次数极多」及其权重 |
| `Rule_name_QJF077` / `Rule_weight_QJF077` | 命中规则「在非银机构申请次数较多」及其权重 |
| `Rule_name_QJF078` / `Rule_weight_QJF078` | 命中规则「在银行机构申请次数极多」及其权重 |
| `Rule_name_QJF079` / `Rule_weight_QJF079` | 命中规则「在银行机构申请次数较多」及其权重 |

`Rule_name_*` / `Rule_weight_*` 仅在命中对应规则时出现。

申请行为变量命名规则：`als_<时间窗口>_<查询维度>_<机构/产品类型>[_<细分>]_<指标>`。

时间窗口：

| 代码 | 含义 |
| --- | --- |
| `d7` | 近 7 天 |
| `d15` | 近 15 天 |
| `m1` | 近 1 个月 |
| `m3` | 近 3 个月 |
| `m6` | 近 6 个月 |
| `m12` | 近 12 个月 |
| `fst` | 距最早一次申请 |
| `lst` | 距最近一次申请 |

查询维度：

| 代码 | 含义 |
| --- | --- |
| `id` | 按身份证号查询 |
| `cell` | 按手机号查询 |

按产品类型统计（形如 `als_m3_id_pdl_allnum`）：

| 代码 | 含义 |
| --- | --- |
| `pdl` | 线上小额现金贷 |
| `caon` | 线上现金分期 |
| `caoff` | 线下现金分期 |
| `coon` | 线上消费分期 |
| `cooff` | 线下消费分期 |
| `rel` | 信用卡（类信用卡） |
| `af` | 汽车金融 |
| `oth` | 其他 |

按机构类型统计（形如 `als_m6_cell_nbank_cons_orgnum`）：

| 代码 | 含义 |
| --- | --- |
| `bank` | 银行机构 |
| `bank_tra` | 银行机构–传统银行 |
| `bank_ret` | 银行机构–网络零售银行 |
| `nbank` | 非银机构 |
| `nbank_p2p` | 非银机构–p2p 机构 |
| `nbank_mc` | 非银机构–小贷机构 |
| `nbank_ca` | 非银机构–现金类分期机构 |
| `nbank_cf` | 非银机构–消费类分期机构 |
| `nbank_com` | 非银机构–代偿类分期机构 |
| `nbank_nsloan` | 非银机构–持牌网络小贷机构 |
| `nbank_autofin` | 非银机构–持牌汽车金融机构 |
| `nbank_sloan` | 非银机构–持牌小贷机构 |
| `nbank_cons` | 非银机构–持牌消费金融机构 |
| `nbank_finlea` | 非银机构–持牌融资租赁机构 |
| `nbank_oth` / `nbank_else` | 非银机构–其他 |
| `bank_week` / `nbank_week` | 在银行 / 非银机构**周末**申请 |
| `bank_night` / `nbank_night` | 在银行 / 非银机构**夜间**申请 |
| （省略机构类型） | 全部机构（如 `als_m3_id_max_inteday`） |

指标：

| 代码 | 含义 |
| --- | --- |
| `allnum` | 申请次数 |
| `orgnum` | 申请机构数 |
| `selfnum` | 在本机构的申请次数 |
| `tot_mons` | 有申请记录的月份数 |
| `avg_monnum` | 平均每月申请次数（按有申请的月份平均） |
| `max_monnum` / `min_monnum` | 最大 / 最小月申请次数 |
| `max_inteday` / `min_inteday` | 申请最大 / 最小间隔天数 |
| `inteday`（`fst` / `lst` 窗口） | 距最早 / 最近一次申请的间隔天数 |
| `consnum`（`lst` 窗口） | 最近开始连续申请的次数 |
| `csinteday`（`lst` 窗口） | 最近开始连续申请的持续天数 |

举例：

| 字段 | 含义 |
| --- | --- |
| `als_m3_id_pdl_allnum` | 按身份证号查询，近 3 个月申请线上小额现金贷的次数 |
| `als_m6_cell_nbank_orgnum` | 按手机号查询，近 6 个月在非银机构申请机构数 |
| `als_m12_id_bank_night_allnum` | 按身份证号查询，近 12 个月在银行机构夜间申请次数 |
| `als_m12_id_avg_monnum` | 按身份证号查询，近 12 个月平均每月申请次数（有申请月份平均） |
| `als_fst_id_nbank_inteday` | 按身份证号查询，距最早在非银机构申请的间隔天数 |
| `als_lst_cell_bank_consnum` | 按手机号查询，最近开始在银行机构连续申请的次数 |

说明：`result.range` 的具体字段可能随数据能力升级扩展，本服务保证完整透出。建议调用方按「存在即解析、缺失即忽略」的方式做容错处理，避免因后续新增字段导致解析失败。

解析示例（JavaScript）：

```
const range = JSON.parse(resp.body.result.range);
const decision = range.Rule_final_decision;          // Accept / Reject / Review
const score = Number(range.Rule_final_weight || 0);  // 最终规则评分
const hits = Object.keys(range)
  .filter(k => k.startsWith("Rule_name_"))
  .map(k => range[k]);                               // 命中规则名称列表
const pdl3m = Number(range.als_m3_id_pdl_allnum || 0);
console.log(decision, score, hits, pdl3m);
```

### 3.2 计费查询数查询（扩展接口）

查询本账户累计计费查询数与调用次数，用于客户侧自助监控与对账。不返回额度上限/剩余量（本服务无额度限制）。

| 项目 | 内容 |
| --- | --- |
| 路径 | `GET /v1/openapi/zlx/quotaFXBL` |
| 完整地址 | `http://aiszcloud.cn:8080/v1/openapi/zlx/quotaFXBL`（或 `http://aiszcloud.com.cn:8080/v1/openapi/zlx/quotaFXBL`） |
| 鉴权 | 与主接口一致（请求体中携带 `appKey` + `sign` 信封；`body` 可为 `{}`，此时 `sign = MD5(appSecret)`） |

响应示例

```
{
  "errorCode": "0",
  "errorMsg": "success",
  "status": "ACTIVE",
  "serviceUsed": 1280,
  "totalCalls": 1280
}
```

| 参数 | 说明 |
| --- | --- |
| status | 账户状态（ACTIVE/SUSPENDED 等） |
| serviceUsed | 累计计费查询次数（`code = 001` 与 `code = 999` 均计入） |
| totalCalls | 累计有效查询次数（含查得与查无；不含被网关拦截的请求） |

说明：无任何额度上限拦截，仅做计费次数统计。本服务对查得与查无同等计费，故 `serviceUsed` 与 `totalCalls` 在正常情况下一致；对账请以 `serviceUsed` 为准。

### 3.3 健康检查

| 项目 | 内容 |
| --- | --- |
| 路径 | `GET /healthz` |
| 完整地址 | `http://aiszcloud.cn:8080/healthz`（或 `http://aiszcloud.com.cn:8080/healthz`） |
| 鉴权 | 无 |
| 响应 | HTTP 200，纯文本 `ok` |

## 四、返回码说明

### 4.1 网关返回码 head.errorCode

| errorCode | 含义 | 典型原因 |
| --- | --- | --- |
| 0 | 成功 | 调用成功（业务结果见 `body.code`） |
| 505001 | appKey 异常 | 缺少或非法 appKey |
| 505004 | 账户信息不存在 | appKey 未匹配到账户 |
| 505007 | 服务尚未开通 | 账户停用 / 过期 / 未开通 |
| 505002 | 账号信息异常 | 签名校验失败 |
| 505003 | 产品编号异常 | 保留 |
| 505062 | 数据请求异常 | 参数非法 / 服务处理异常 / 超时未决 / 系统错误（默认错误码） |

服务处理异常（账户/参数/系统/超时未决等）统一返回 `505062`，不计费，并经异步复查兜底。

### 4.2 业务结果码 body.code（仅 errorCode = 0 时）

| code | 含义 | 是否计费 |
| --- | --- | --- |
| 001 | 查得数据 | 计费 |
| 999 | 查无结果 | 计费 |

注：本服务按有效查询次数计费。只要本次查询产出了确定结论——无论是查得数据（`001`）还是确认无可用记录（`999`）——均计费一次。仅当返回 `head.errorCode` 非 0（未能产出结论）时不计费。

## 五、计费说明

- `body.code = 001`（查得数据）与 `body.code = 999`（查无结果）**均计入计费查询次数并对客户计费**。
- 本服务按**有效查询次数**计费：查无结果同样消耗一次查询并产出确定结论。
- 网关级错误（`head.errorCode` 非 0：鉴权失败、参数非法、系统异常等）一律不计费。
- 计费以最终落库的台账为准，超时未决请求会经异步复查/对账裁定状态，不会重复计费。
- 计费次数可随时通过「计费查询数查询（3.2）」接口自助核对。

## 六、使用手册（接入与最佳实践）

### 6.1 接入流程

1. 向商务申请账户，获取 `appKey`、`appSecret`，并提供贵司外网出口 IP 用于加白；服务地址见「1.2 接入须知」。
2. 按第二章实现加签，先在测试环境联调，再切正式环境。
3. 上线后通过「计费查询数查询（3.2）」接口监控调用量与计费次数。

### 6.2 幂等与重试

- 客户端建议为每笔查询设置合理超时（≥ 5 秒）。
- 收到网络超时/无响应时可安全重试：网关基于内部流水号做幂等，不会因重试重复计费。
- 请勿对已明确返回 `head.errorCode` 的请求做无差别重试（如参数错误 `505062`、鉴权错误 `505001` / `505002` / `505004`），应先修正再发起。

### 6.3 错误处理建议

| 现象 | 排查方向 |
| --- | --- |
| 505001 / 505004 | 检查 appKey 是否正确、是否用错环境账户 |
| 505002 | 检查签名算法（排序/空值剔除/UTF-8/小写 hex） |
| 505007 | 联系商务确认账户状态与有效期 |
| 505062 | 检查 name / idCard / mobile 是否齐全合法；若入参正常仍持续出现，记录 logId 联系我方 |

任何异常排查请一并提供响应中的 `head.logId`，便于我方全链路定位。

### 6.4 联调自检清单

- [ ] 服务地址（aiszcloud.cn:8080 或 aiszcloud.com.cn:8080）、appKey、appSecret、环境匹配无误，出口 IP 已报我方加白
- [ ] 待签名串严格按 ASCII 升序拼接、剔除空值、UTF-8、MD5 小写

- [ ] name / idCard / mobile 三项均以明文传入且齐全合法
- [ ] 能正确解析 `head.errorCode` 与 `body.code` 两级状态
- [ ] 已知悉 `999` 查无结果同样计费，业务侧计费预估按有效查询次数测算
- [ ] 已实现对 `result.range` 字符串的二次 `JSON.parse` 解析与字段容错
- [ ] 已实现超时重试（依赖幂等，不重复计费）

## 附录：术语表

| 术语 | 说明 |
| --- | --- |
| appKey | 公开账户标识，随请求明文传输 |
| appSecret | 加签密钥，仅本地保存用于计算 sign，切勿泄露或随请求传输 |
| logId | 全链路追踪 ID（= `head.logId`），排障/对账唯一凭据 |
| range | 风险变量结果的 JSON 字符串，需二次解析（结构见 3.1.5） |
| 有效查询 | 产出了确定结论的查询（`body.code = 001` 或 `999`），即本服务的计费单位 |
| 申请行为变量 | `als_` 前缀字段，按时间窗口 × 查询维度 × 机构/产品类型 × 指标组合命名 |